

AI READY PREMIUM KIT

Glossario dei 64 Errori Comuni

Per non restare mai bloccato: riconosci l'errore, capisci la causa, correggilo

Materiale bonus di "Python da Zero con Google Colab" - Tania Cilio

Come usare questo glossario

Ogni principiante incontra sempre gli stessi errori. La buona notizia? Sono pochi, sono prevedibili, e una volta che li conosci non ti fermano più. Questo glossario raccoglie gli errori più comuni in Python, spiegati in modo semplice.

Per ogni errore trovi: il sintomo (cosa vedi a schermo), la causa (perché succede), il codice sbagliato, il codice corretto e una regola pratica per non ripeterlo. Quando ti blocchi, cerca qui il messaggio che vedi: quasi sicuramente è in questa lista.

Gli errori sono raggruppati per argomento. Tienilo aperto di fianco mentre programmi: è il tuo pronto soccorso.

Errori: Sintassi

1. Manca il due punti dopo if/for/while/def

Sintomo: *SyntaxError: expected ':'*

Causa: Le istruzioni che aprono un blocco (if, for, while, def) finiscono sempre con i due punti.

Sbagliato:

```
if eta >= 18  
    print("ok")
```

Corretto:

```
if eta >= 18:  
    print("ok")
```

Regola: Ogni volta che apri un blocco, controlla di aver messo i due punti alla fine della riga.

2. Indentazione mancante o sbagliata

Sintomo: *IndentationError: expected an indented block*

Causa: Il codice dentro un blocco (dopo i due punti) deve essere rientrato, di solito con 4 spazi.

Sbagliato:

```
if eta >= 18:  
print("ok")
```

Corretto:

```
if eta >= 18:  
    print("ok")
```

Regola: Dopo i due punti, vai a capo e rientra di 4 spazi. Usa sempre lo stesso numero di spazi.

3. Parentesi non chiusa

Sintomo: *SyntaxError: '(' was never closed*

Causa: Hai aperto una parentesi e non l'hai chiusa.

Sbagliato:

```
print("Ciao"
```

Corretto:

```
print("Ciao")
```

Regola: Per ogni parentesi aperta deve essercene una chiusa. Conta le coppie.

4. Virgolette non chiuse o miste

Sintomo: *SyntaxError: unterminated string literal*

Causa: Una stringa deve aprirsi e chiudersi con lo stesso tipo di virgolette.

Sbagliato:

```
print("Ciao')
```

Corretto:

```
print("Ciao")
```

Regola: Apri e chiudi con lo stesso tipo di virgolette: o entrambe doppie o entrambe singole.

5. Uso di = invece di == nei confronti

Sintomo: *SyntaxError oppure logica sbagliata*

Causa: = assegna un valore, == confronta due valori. Nelle condizioni serve ==.

Sbagliato:

```
if eta = 18:  
    print("ok")
```

Corretto:

```
if eta == 18:  
    print("ok")
```

Regola: Ricorda: un uguale assegna, due uguali confrontano.

6. Virgola decimale invece del punto

Sintomo: *Crea una tupla invece di un numero*

Causa: In Python i decimali si scrivono col punto. La virgola separa elementi e crea una tupla.

Sbagliato:

```
prezzo = 19,99
print(prezzo * 2)
```

Corretto:

```
prezzo = 19.99
print(prezzo * 2)
```

Regola: I decimali vogliono il punto: 19.99, mai 19,99.

7. Dimenticare la f nella f-string

Sintomo: *Le graffe vengono stampate così come sono*

Causa: Per inserire variabili dentro il testo serve la f prima delle virgolette.

Sbagliato:

```
nome = "Anna"
print("Ciao {nome}")
```

Corretto:

```
nome = "Anna"
print(f"Ciao {nome}")
```

Regola: Se usi le graffe per le variabili, metti sempre la f davanti alle virgolette.

Errori: Variabili

8. Variabile usata prima di crearla

Sintomo: *NameError: name '...' is not defined*

Causa: Stai usando una variabile che non hai ancora definito.

Sbagliato:

```
print(totale)
totale = 100
```

Corretto:

```
totale = 100
print(totale)
```

Regola: Crea sempre la variabile (con =) prima di usarla.

9. Nome scritto in modo diverso

Sintomo: *NameError: name '...' is not defined*

Causa: Hai definito la variabile con un nome e la usi con un altro (anche solo una lettera diversa).

Sbagliato:

```
nome_utente = "Anna"
print(nomeutente)
```

Corretto:

```
nome_utente = "Anna"
print(nome_utente)
```

Regola: Scrivi i nomi sempre identici: Python distingue ogni carattere.

10. Maiuscole e minuscole confuse

Sintomo: *NameError: name '...' is not defined*

Causa: Python distingue maiuscole e minuscole: Nome e nome sono due variabili diverse.

Sbagliato:

```
Nome = "Anna"
print(nome)
```

Corretto:

```
Nome = "Anna"
print(Nome)
```

Regola: Usa sempre la stessa capitalizzazione con cui hai definito la variabile.

11. Nome che inizia con un numero

Sintomo: *SyntaxError: invalid decimal literal*

Causa: I nomi di variabile non possono iniziare con un numero.

Sbagliato:

```
1prezzo = 10
```

Corretto:

```
prezzo1 = 10
```

Regola: I nomi possono contenere numeri ma non iniziare con essi. Metti il numero alla fine.

12. Spazio nel nome della variabile

Sintomo: *SyntaxError: invalid syntax*

Causa: I nomi di variabile non possono contenere spazi.

Sbagliato:

```
nome utente = "Anna"
```

Corretto:

```
nome_utente = "Anna"
```

Regola: Sostituisci gli spazi con l'underscore: snake_case.

13. Usare una parola riservata come nome

Sintomo: *SyntaxError*

Causa: Parole come if, for, class, print non si possono usare come nomi di variabile.

Sbagliato:

```
if = 5
```

Corretto:

```
numero_if = 5
```

Regola: Non usare le parole chiave di Python come nomi. Scegli un nome descrittivo diverso.

14. Sovrascrivere una funzione utile

Sintomo: *TypeError quando poi la richiami*

Causa: Se chiami una variabile come una funzione (es. list, sum), perdi quella funzione.

Sbagliato:

```
list = [1, 2, 3]
nuova = list((4, 5))
```

Corretto:

```
numeri = [1, 2, 3]
nuova = list((4, 5))
```

Regola: Non chiamare le variabili come funzioni note: list, sum, str, max, min, type.

Errori: Tipi

15. Sommare testo e numero

Sintomo: *TypeError: can only concatenate str (not "int") to str*

Causa: Non puoi unire direttamente una stringa e un numero con +.

Sbagliato:

```
eta = 30
print("Ho " + eta + " anni")
```

Corretto:

```
eta = 30
print("Ho " + str(eta) + " anni")
```

Regola: Per unire testo e numero, converti il numero con str() oppure usa una f-string.

16. Dimenticare int() sull'input

Sintomo: *TypeError o risultato sbagliato*

Causa: input() restituisce sempre testo: per i calcoli va convertito.

Sbagliato:

```
eta = input("Età: ")
print(eta + 1)
```

Corretto:

```
eta = int(input("Età: "))
print(eta + 1)
```

Regola: Avvolgi input() in int() o float() quando ti serve un numero.

17. Convertire in int un decimale scritto come testo

Sintomo: *ValueError: invalid literal for int()*

Causa: int() non accetta una stringa con il punto decimale.

Sbagliato:

```
prezzo = int("19.99")
```

Corretto:

```
prezzo = float("19.99")
```

Regola: Per i decimali usa float(), non int(). int() vuole numeri interi.

18. Convertire in numero un testo non numerico

Sintomo: *ValueError: invalid literal for int()*

Causa: int() o float() falliscono se la stringa non è un numero.

Sbagliato:

```
n = int("ciao")
```

Corretto:

```
# proteggi con try/except
try:
    n = int("ciao")
except ValueError:
    n = 0
```

Regola: Quando converti un input dell'utente, proteggi la conversione con try/except.

19. Moltiplicare due input senza convertire

Sintomo: *I testi si ripetono invece di moltiplicarsi*

Causa: '3' * 2 dà '33' (ripete il testo), non 6.

Sbagliato:

```
a = input("a: ")
b = input("b: ")
print(a * b)
```

Corretto:

```
a = int(input("a: "))
b = int(input("b: "))
print(a * b)
```

Regola: Converti SEMPRE in numero gli input prima di farci operazioni matematiche.

20. Confrontare numero e testo

Sintomo: *Confronto sempre falso*

Causa: "10" (testo) non è uguale a 10 (numero).

Sbagliato:

```
risposta = input("Numero: ")
if risposta == 10:
    print("ok")
```

Corretto:

```
risposta = int(input("Numero: "))
if risposta == 10:
    print("ok")
```

Regola: Confronta cose dello stesso tipo: converti il testo in numero prima di confrontarlo con un numero.

21. True/False scritti minuscoli

Sintomo: *NameError: name 'true' is not defined*

Causa: In Python si scrivono True e False con la maiuscola.

Sbagliato:

```
attivo = true
```

Corretto:

```
attivo = True
```

Regola: I booleani sono True e False, con la prima lettera maiuscola.

Errori: Stringhe

22. Pensare che .upper() cambi la variabile

Sintomo: La variabile resta invariata

Causa: I metodi delle stringhe restituiscono un nuovo testo, non modificano l'originale.

Sbagliato:

```
nome = "anna"  
nome.upper()  
print(nome)
```

Corretto:

```
nome = "anna"  
nome = nome.upper()  
print(nome)
```

Regola: Per salvare il risultato, riassegna: nome = nome.upper().

23. Indice di stringa fuori limite

Sintomo: *IndexError: string index out of range*

Causa: Stai chiedendo un carattere che non esiste.

Sbagliato:

```
parola = "ciao"  
print(parola[10])
```

Corretto:

```
parola = "ciao"  
print(parola[-1])
```

Regola: Gli indici vanno da 0 a len-1. Per l'ultimo carattere usa [-1].

24. Dimenticare lo spazio quando unisci

Sintomo: Le parole risultano attaccate

Causa: Il + non aggiunge spazi: li devi mettere tu.

Sbagliato:

```
nome = "Anna"  
cognome = "Rossi"  
print(nome + cognome)
```

Corretto:

```
print(nome + " " + cognome)
```

Regola: Aggiungi uno spazio esplicito tra le parole, o usa una f-string.

25. Apostrofo dentro stringa con apici singoli

Sintomo: *SyntaxError*

Causa: L'apostrofo chiude la stringa aperta con apici singoli.

Sbagliato:

```
frase = 'L'amico'
```

Corretto:

```
frase = "L'amico"
```

Regola: Se il testo contiene un apostrofo, racchiudilo tra virgolette doppie.

26. Credere che strip() tolga gli spazi interni

Sintomo: Gli spazi in mezzo restano

Causa: strip() toglie solo gli spazi ai bordi, non quelli interni.

Sbagliato:

```
testo = " a b "  
print(testo.strip())
```

Corretto:

```
testo = " a b "  
print(testo.strip().replace(" ", " "))
```

Regola: strip() pulisce solo inizio e fine. Per gli spazi interni usa replace().

Errori: Liste

27. Indice di lista fuori limite

Sintomo: *IndexError: list index out of range*

Causa: Stai accedendo a una posizione che non esiste nella lista.

Sbagliato:

```
frutti = ["mela", "pera"]  
print(frutti[5])
```

Corretto:

```
frutti = ["mela", "pera"]  
print(frutti[-1])
```

Regola: Ricorda: gli indici partono da 0. L'ultimo elemento è [-1].

28. Pensare che il primo indice sia 1

Sintomo: *Salti il primo elemento*

Causa: In Python il primo elemento è all'indice 0, non 1.

Sbagliato:

```
frutti = ["mela", "pera", "kiwi"]  
print(frutti[1]) # voglio il primo
```

Corretto:

```
frutti = ["mela", "pera", "kiwi"]  
print(frutti[0]) # primo elemento
```

Regola: Il conteggio degli indici parte da 0: il primo elemento è [0].

29. remove() di un elemento che non c'è

Sintomo: *ValueError: list.remove(x): x not in list*

Causa: remove() fallisce se l'elemento non è nella lista.

Sbagliato:

```
frutti = ["mela"]  
frutti.remove("pera")
```

Corretto:

```
frutti = ["mela"]  
if "pera" in frutti:  
    frutti.remove("pera")
```

Regola: Controlla con 'in' che l'elemento esista prima di rimuoverlo.

30. Modificare una lista mentre la scorri

Sintomo: *Comportamento imprevedibile*

Causa: Aggiungere o togliere elementi durante un for sulla stessa lista crea problemi.

Sbagliato:

```
numeri = [1, 2, 3, 4]  
for n in numeri:  
    if n % 2 == 0:  
        numeri.remove(n)
```

Corretto:

```
numeri = [1, 2, 3, 4]  
numeri = [n for n in numeri if n % 2 != 0]
```

Regola: Non modificare una lista mentre la scorri: crea una nuova lista filtrata.

Errori: Dizionari

31. Chiave inesistente con parentesi quadre

Sintomo: *KeyError: '...'*

Causa: Accedere con [] a una chiave che non esiste fa crashare il programma.

Sbagliato:

```
persona = {"nome": "Anna"}  
print(persona["eta"])
```

Corretto:

```
persona = {"nome": "Anna"}
```



```
print(persona.get("eta", "N/D"))
```

Regola: Usa `.get()` per leggere chiavi che potrebbero non esistere: restituisce un default invece di un errore.

32. Confondere parentesi quadre e tonde

Sintomo: *TypeError*

Causa: Ai valori di un dizionario si accede con le quadre `[]`, non le tonde `()`.

Sbagliato:

```
persona = {"nome": "Anna"}
print(persona("nome"))
```

Corretto:

```
persona = {"nome": "Anna"}
print(persona["nome"])
```

Regola: Dizionari e liste usano le parentesi quadre per accedere agli elementi.

33. Scorrere il dizionario aspettandosi i valori

Sintomo: *Ottieni le chiavi, non i valori*

Causa: Un `for` diretto sul dizionario restituisce le chiavi.

Sbagliato:

```
persona = {"nome": "Anna", "eta": 30}
for x in persona:
    print(x) # voglio i valori
```

Corretto:

```
persona = {"nome": "Anna", "eta": 30}
for chiave, valore in persona.items():
    print(valore)
```

Regola: Per avere chiavi e valori insieme usa `.items()`.

Errori: Cicli

34. range che si ferma troppo presto

Sintomo: *Manca l'ultimo numero*

Causa: `range(1, 10)` arriva a 9: l'estremo destro è escluso.

Sbagliato:

```
for i in range(1, 10):
    print(i) # voglio fino a 10
```

Corretto:

```
for i in range(1, 11):
    print(i)
```

Regola: `range(a, b)` arriva fino a `b-1`. Per includere `b`, scrivi `b+1`.

35. Ciclo while infinito

Sintomo: *Il programma non si ferma mai*

Causa: La condizione del `while` resta sempre vera perché non aggiorni la variabile.

Sbagliato:

```
# n non cambia mai -> ciclo infinito
# n = 1
# while n <= 5:
#     print(n)
```

Corretto:

```
n = 1
while n <= 5:
    print(n)
    n += 1
```

Regola: Dentro un `while`, aggiorna sempre la variabile della condizione, o non finirà mai.

36. Accumulatore azzerato dentro il ciclo

Sintomo: *Risultato sempre uguale all'ultimo elemento*

Causa: Se metti `totale = 0` dentro il `for`, si azzerà a ogni giro.

Sbagliato:

```
numeri = [1, 2, 3]
for n in numeri:
    totale = 0
    totale += n
print(totale)
```

Corretto:

```
numeri = [1, 2, 3]
totale = 0
for n in numeri:
    totale += n
print(totale)
```

Regola: L'accumulatore va inizializzato PRIMA del ciclo, non dentro.

Errori: Condizioni

37. Confronto con and/or scritto male

Sintomo: *Logica sbagliata o errore*

Causa: Ogni confronto deve essere completo: `eta > 18 and eta < 65`, non `eta > 18 and < 65`.

Sbagliato:

```
eta = 30
if eta > 18 and < 65:
    print("ok")
```

Corretto:

```
eta = 30
if eta > 18 and eta < 65:
    print("ok")
```

Regola: Ripeti la variabile in ogni confronto: `'eta > 18 and eta < 65'`.

38. elif senza if iniziale

Sintomo: *SyntaxError*

Causa: `elif` deve sempre seguire un `if`.

Sbagliato:

```
elif voto >= 60:
    print("ok")
```

Corretto:

```
if voto >= 60:
    print("ok")
```

Regola: Una catena di condizioni inizia sempre con `if`, poi eventuali `elif`, poi `else`.

39. Troppi if separati invece di elif

Sintomo: *Più rami eseguiti del previsto*

Causa: Con `if` separati, più blocchi possono attivarsi; con `elif` solo il primo vero.

Sbagliato:

```
voto = 95
if voto >= 60: print("Sufficiente")
if voto >= 90: print("Ottimo")
```

Corretto:

```
voto = 95
if voto >= 90:
    print("Ottimo")
elif voto >= 60:
    print("Sufficiente")
```

Regola: Quando i casi si escludono a vicenda, usa `elif`, non tanti `if`.

Errori: Funzioni

40. Chiamare la funzione senza parentesi

Sintomo: *Non viene eseguita*

Causa: Senza le parentesi, ti riferisci alla funzione ma non la chiami.

Sbagliato:

```
def saluta():  
    print("Ciao")  
saluta
```

Corretto:

```
def saluta():  
    print("Ciao")  
saluta()
```

Regola: Per eseguire una funzione servono le parentesi: saluta().

41. Dimenticare un argomento obbligatorio

Sintomo: *TypeError: missing 1 required positional argument*

Causa: La funzione ha un parametro obbligatorio che non hai passato.

Sbagliato:

```
def saluta(nome):  
    print(f"Ciao {nome}")  
saluta()
```

Corretto:

```
def saluta(nome):  
    print(f"Ciao {nome}")  
saluta("Anna")
```

Regola: Passa tutti gli argomenti che la funzione richiede.

42. Confondere print e return

Sintomo: *La funzione restituisce None*

Causa: print() mostra ma non restituisce; per riusare il valore serve return.

Sbagliato:

```
def doppio(n):  
    print(n * 2)  
x = doppio(5)  
print(x + 1)
```

Corretto:

```
def doppio(n):  
    return n * 2  
x = doppio(5)  
print(x + 1)
```

Regola: Se vuoi riutilizzare il risultato, usa return, non print.

43. Parametro con default prima di uno senza

Sintomo: *SyntaxError: non-default argument follows default argument*

Causa: I parametri con valore di default vanno dopo quelli senza.

Sbagliato:

```
def f(iva=0.22, prezzo):  
    return prezzo
```

Corretto:

```
def f(prezzo, iva=0.22):  
    return prezzo
```

Regola: Metti prima i parametri obbligatori, poi quelli con default.

44. Usare una variabile interna fuori dalla funzione

Sintomo: *NameError*

Causa: Le variabili create dentro una funzione non esistono fuori.

Sbagliato:

```
def calcola():  
    risultato = 42  
calcola()  
print(risultato)
```

Corretto:

```
def calcola():  
    return 42  
risultato = calcola()  
print(risultato)
```

Regola: Per usare un valore fuori dalla funzione, restituiscilo con return.

Errori: File

45. Aprire un file che non esiste in lettura

Sintomo: *FileNotFoundError*

Causa: Stai cercando di leggere un file che non c'è.

Sbagliato:

```
# open("inesistente.txt", "r")
```

Corretto:

```
import os
if os.path.exists("file.txt"):
    open("file.txt", "r")
```

Regola: Controlla che il file esista, o gestisci l'errore con try/except FileNotFoundError.

46. Dimenticare encoding con gli accenti

Sintomo: *Caratteri strani o UnicodeError*

Causa: Senza encoding utf-8, gli accenti italiani possono rompersi.

Sbagliato:

```
open("f.txt", "w")
```

Corretto:

```
open("f.txt", "w", encoding="utf-8")
```

Regola: Usa sempre encoding="utf-8" quando apri file con testo italiano.

47. Dimenticare l'a-capo scrivendo righe

Sintomo: *Le righe si attaccano*

Causa: write() non va a capo da solo: devi aggiungere backslash-n.

Sbagliato:

```
# f.write("riga1")
# f.write("riga2")
```

Corretto:

```
# f.write("riga1\n")
# f.write("riga2\n")
```

Regola: Aggiungi \n alla fine di ogni riga che scrivi su file.

Errori: Input

48. Non pulire l'input dell'utente

Sintomo: *Confronti che falliscono per spazi*

Causa: L'utente può aggiungere spazi: 'si ' non è uguale a 'si'.

Sbagliato:

```
r = input("si/no: ")
if r == "si":
    print("ok")
```

Corretto:

```
r = input("si/no: ").strip().lower()
if r == "si":
    print("ok")
```

Regola: Pulisci l'input con .strip() e uniforma con .lower() prima di confrontarlo.

Errori: Logica

49. Priorità degli operatori dimenticata

Sintomo: *Risultato matematico sbagliato*

Causa: Senza parentesi, moltiplicazione e divisione vengono prima di somma e sottrazione.

Sbagliato:

```
media = 10 + 20 + 30 / 3
print(media)
```

Corretto:

```
media = (10 + 20 + 30) / 3
print(media)
```

Regola: Usa le parentesi per controllare l'ordine dei calcoli.

50. Divisione per zero

Sintomo: *ZeroDivisionError: division by zero*

Causa: Non si può dividere per zero.

Sbagliato:

```
a = 10
b = 0
print(a / b)
```

Corretto:

```
a = 10
b = 0
if b != 0:
    print(a / b)
else:
    print("Divisore zero")
```

Regola: Controlla che il divisore non sia zero prima di dividere.

51. Confondere / e //

Sintomo: *Tipo di risultato inatteso*

Causa: / dà sempre un decimale, // dà il quoziente intero.

Sbagliato:

```
print(10 / 3) # voglio 3
```

Corretto:

```
print(10 // 3) # quoziente intero
```

Regola: Usa // per il quoziente intero, / per la divisione decimale.

52. Arrotondamenti dei decimali (float)

Sintomo: *Risultati tipo 0.30000000000000004*

Causa: I float hanno piccole imprecisioni: è normale in tutti i linguaggi.

Sbagliato:

```
print(0.1 + 0.2)
```

Corretto:

```
print(round(0.1 + 0.2, 2))
```

Regola: Per mostrare i decimali usa round() o la formattazione :.2f.

53. Confronto tra float con ==

Sintomo: *Confronto inaspettatamente falso*

Causa: A causa delle imprecisioni, due float 'uguali' possono risultare diversi.

Sbagliato:

```
print(0.1 + 0.2 == 0.3)
```

Corretto:

```
print(round(0.1 + 0.2, 10) == round(0.3, 10))
```

Regola: Per confrontare float, arrotonda entrambi o verifica che la differenza sia piccolissima.

54. Dimenticare che int() tronca, non arrotonda

Sintomo: *Perdi la parte decimale*

Causa: int(3.9) dà 3, non 4: taglia i decimali.

Sbagliato:

```
print(int(3.9)) # mi aspetto 4
```

Corretto:

```
print(round(3.9)) # arrotonda a 4
```

Regola: Per arrotondare usa round(); int() taglia e basta la parte decimale.

55. Usare 'is' invece di == per i valori

Sintomo: *Confronto inaffidabile*

Causa: 'is' confronta l'identità, non il valore. Per i valori usa ==.

Sbagliato:

```
a = 1000
b = 1000
print(a is b)
```

Corretto:

```
a = 1000
b = 1000
print(a == b)
```

Regola: Per confrontare valori usa ==. Usa 'is' solo con None (x is None).

56. Concatenare confronti in modo errato

Sintomo: *Logica sbagliata*

Causa: Per verificare un intervallo, Python permette $0 \leq x \leq 10$.

Sbagliato:

```
x = 5
if 0 <= x and 10:
    print("ok")
```

Corretto:

```
x = 5
if 0 <= x <= 10:
    print("ok")
```

Regola: Per un intervallo scrivi $0 \leq x \leq 10$, oppure usa due confronti completi con and.

Errori: Vari

57. Mescolare tabulazioni e spazi nell'indentazione

Sintomo: *TabError: inconsistent use of tabs and spaces*

Causa: Mischiare TAB e spazi per rientrare confonde Python.

Sbagliato:

```
# riga con tab e riga con spazi nello stesso blocco
```

Corretto:

```
# usa SEMPRE 4 spazi (mai il tasto Tab misto a spazi)
```

Regola: Configura l'editor per usare 4 spazi e non mischiare mai con il tasto Tab.

58. Copiare virgolette 'curve' da Word

Sintomo: *SyntaxError: invalid character*

Causa: Le virgolette tipografiche di Word non sono valide in Python.

Sbagliato:

```
# print("Ciao") # virgolette curve
```

Corretto:

```
print("Ciao") # virgolette dritte
```

Regola: Scrivi il codice in Colab o in un editor di codice, non in Word: usa virgolette dritte.

59. Dimenticare di rieseguire la cella in Colab

Sintomo: *Usi un valore vecchio*

Causa: Se modifichi una cella e non la riesegui, Colab usa ancora il valore precedente.

Sbagliato:

```
# modifichi x ma non riesegui la cella
```

Corretto:

```
# premi Ctrl+Invio per rieseguire la cella dopo ogni modifica
```

Regola: In Colab, riesegui la cella (Ctrl+Invio) ogni volta che cambi qualcosa.

60. Eseguire le celle in disordine in Colab

Sintomo: *NameError o risultati strani*

Causa: Le celle vanno eseguite dall'alto verso il basso: l'ordine conta.

Sbagliato:

```
# esegui prima la cella che usa x, poi quella che crea x
```

Corretto:

```
# esegui le celle in ordine, dalla prima all'ultima
```

Regola: In Colab esegui le celle in ordine. Se hai dubbi: Runtime > Riavvia ed esegui tutto.

61. Confondere = e == ancora una volta (logica)

Sintomo: *Assegnazione invece di confronto*

Causa: Nelle condizioni l'errore tra = e == è così comune da meritare due voci.

Sbagliato:

```
# if x = 5: -> errore
```

Corretto:

```
if x == 5:  
    print("ok")
```

Regola: Nelle condizioni serve sempre il doppio uguale ==

62. Stampare invece di restituire in una catena di funzioni

Sintomo: *None inatteso nei calcoli*

Causa: Se una funzione stampa invece di restituire, le funzioni che la usano ricevono None.

Sbagliato:

```
def meta(n):  
    print(n / 2)  
    risultato = meta(10) + 1
```

Corretto:

```
def meta(n):  
    return n / 2  
risultato = meta(10) + 1
```

Regola: Nelle funzioni che servono ad altre funzioni, usa return.

63. Dimenticare i due punti nella definizione del dizionario

Sintomo: *SyntaxError*

Causa: Nei dizionari, chiave e valore sono separati dai due punti.

Sbagliato:

```
persona = {"nome" "Anna"}
```

Corretto:

```
persona = {"nome": "Anna"}
```

Regola: Nei dizionari usa chiave: valore, separati dai due punti.

64. Usare append con due argomenti

Sintomo: *TypeError: append() takes exactly one argument*

Causa: append() aggiunge un solo elemento per volta.

Sbagliato:

```
lista = []  
lista.append("a", "b")
```

Corretto:

```
lista = []  
lista.append("a")  
lista.append("b")
```

Regola: append() vuole un solo elemento. Per aggiungerne più di uno usa extend() o più append().

Hai sbagliato e hai trovato l'errore qui? Allora ricordati la cosa più importante: sbagliare è il modo in cui si impara a programmare. Ogni errore capito è un passo avanti. Nessuno nasce imparato.